Embedded and Intelligent Systems

MASTER THESIS

Data-Driven Activity Recognition and Energy Consumption Forecasting for Heavy-Duty Vehicles

Harrin John Ignatious

Supervisor : Yuantao Fan

Halmstad University, March 20, 2026 –version 4.0

Harrin JohnIgnatious : *Data-Driven Activity Recognition and Energy Consumption Forecasting for Heavy-Duty Vehicles* , © march 2026

CONTENTS

1 INTRODUCTION 1
- 1.1 Objective 1
- 1.2 Problem Statement 2
- 1.3 Research Questions 3
- 1.4 Novelty and Contribution 3
- 1.5 Evaluation 4
  - 1.5.1 Activity Discovery Evaluation and Analysis (Segmentation and Clustering) 4
  - 1.5.2 Model Performance Evaluation (Energy Forecasting) 4
  - 1.5.3 Online Learning Evaluation 5

2 LITERATURE REVIEW 7
- 2.1 Background 7
  - 2.1.1 Auxiliary-driven operational modes in vehicle telemetry 7
  - 2.1.2 Unsupervised activity discovery: segmentation and clustering of time series 8
  - 2.1.3 Time-series segmentation and change-point detection 8
  - 2.1.4 Time-series clustering and similarity measures 9
- 2.2 Multi-step forecasting for energy-related signals 9
- 2.3 Online and Continual Learning for Streaming Time Series 10

3 METHOD 11
- 3.1 Model Architecture 11
  - 3.1.1 Cycle Extraction and Resampling 11
  - 3.1.2 Activity Discovery (Segmentation and Clustering) 12
  - 3.1.3 Fixed-Length Windowing 13
  - 3.1.4 Forecasting Models 13
  - 3.1.5 Online Learning Architecture (Prequential Adaptation) 14
- 3.2 Loss Functions 14

4 DATASET DESCRIPTION 17
  - 4.0.1 Dataset 17
  - 4.0.2 Data Preprocessing 17

5 EXPERIMENTAL EVALUATION 19
- 5.1 Training Setup 19
  - 5.1.1 Data and Task Definition 19

6 RESULTS 21
- 6.1 PTO Cycle Extraction Results 21

6.1.1 Examples of Retained and Excluded PTO Cycles 21
6.1.2 Retained (valid) cycle. 22
6.1.3 Excluded (invalid) cycle. 22
6.2 Sub-cycle Segmentation Results 22
6.2.1 Derivative-based segmentation. 22
6.2.2 Matrix Profile 23
6.2.3 Change-Point Detection 24
6.2.4 Window Correlation 24
6.2.5 Energy-Based 25
6.3 Sub-cycle Clustering Results 26
6.3.1 PELT + Feature-based KMeans 26
6.3.2 Change-Point Segmentation + DTW-based Clustering 27
6.3.3 Hybrid Rule-based + KMeans 28
6.3.4 Spectrogram-Based Clustering 28
6.3.5 Step-level KMeans on Fixed-Length Cycle) 29
6.4 Forecasting Results (Regression) 30
6.4.1 Offline Multi-step Forecasting Setup 30
6.5 Online Learning Results 32
6.5.1 LSTM Cycle-wise Test-then-Update 32
6.5.2 MLP-Based Prequential Learning with Replay Buffer 34

BIBLIOGRAPHY 37

LIST OF FIGURES

Figure 1 Overview of the proposed pipeline for auxiliary activity discovery and forecasting. 11
Figure 2 Valid cycle 21
Figure 3 Invalid cycle 22
Figure 4 derivative based 23
Figure 5 Matrix profile 24
Figure 6 Change point detection 24
Figure 7 Window based 25
Figure 8 Energy based 26
Figure 9 change point and K means 27
Figure 10 change point and DTW 27
Figure 11 Hybrid rule based + Kmeans 28
Figure 12 Spectrum based 29
Figure 13 Step level Kmeans on fixed length 30
Figure 14 Mean based 31
Figure 15 LSTM 31
Figure 16 LSTM and attenuation 32
Figure 17 LSTM online 33
Figure 18 LSTM offline 33
Figure 19 MLP online 34
Figure 20 MLP offline 34

LIST OF TABLES

Table 1 Offline forecasting performance on the test set (normalized error metrics). 32

1 INTRODUCTION

Energy consumption forecasting is an important task for heavy-duty vehicles because it supports energy management, trip planning, and overall operational efficiency. In real-world driving, vehicle energy use is influenced not only by traction demand but also by auxiliary systems such as heating, ventilation, and air conditioning (HVAC), thermal management components, compressors, and pumps. The activity of these systems varies with external factors such as weather conditions, as well as operational factors including driving patterns and vehicle usage. Consequently, their contribution to total energy consumption can be significant, and two trips with similar speed and distance may exhibit different energy usage if the auxiliary systems behave differently. Ignoring these auxiliary effects can reduce the accuracy of energy consumption forecasting models.

This thesis focuses on capturing the influence of auxiliary systems in a data-driven manner, organized around two connected tasks: auxiliary activity recognition and energy forecasting. Since explicit labels for auxiliary activity are not available, activity patterns are first discovered from multivariate vehicle signals using unsupervised clustering. The resulting clusters are then interpreted and labeled with support from domain experts, producing a practical set of auxiliary activity categories that reflect real vehicle behavior.

Next, auxiliary activity is inferred from vehicle telemetry using the clustering-derived activity taxonomy and an activity assignment model that maps observations to an activity category. The resulting activity estimate is then incorporated as an additional input, alongside standard vehicle telemetry, to forecast future energy consumption. Explicitly representing auxiliary activity in this way is expected to help the forecasting model better handle variations in auxiliary usage and reduce prediction errors when auxiliary loads contribute significantly to total energy demand. The proposed approach is evaluated by comparing it with a baseline model that forecasts energy directly from telemetry without auxiliary activity information, and the results are analyzed to identify the operating conditions in which the activity-aware approach provides the largest improvement.

1.1 OBJECTIVE

The primary objective of this thesis is to develop and evaluate a practical framework for energy consumption forecasting in heavy-duty

vehicles, with a focus on capturing the impact of auxiliary systems such as HVAC and thermal management using multivariate vehicle telemetry.

Because explicit auxiliary activity labels are not available, the thesis aims to construct an auxiliary activity representation directly from data. This includes defining a clustering-based procedure to discover recurring auxiliary operation patterns and establishing a process for domain-expert validation and naming of the discovered patterns.

Finally, the thesis aims to quantify the value of auxiliary activity information for forecasting. This is done through controlled comparisons between a baseline forecasting model using standard telemetry only and an activity-aware forecasting model that additionally uses the inferred auxiliary activity representation as input, with analysis of when the activity-aware approach provides the largest improvements.

1.2 PROBLEM STATEMENT

A key challenge in this thesis is that auxiliary activity is not available as reliable ground-truth labels in the dataset. This prevents a direct supervised learning approach for auxiliary activity recognition and makes it unclear how to define a consistent auxiliary activity variable that can be used in downstream modeling. As a result, auxiliary activity must be discovered from multivariate vehicle telemetry using unsupervised methods and then validated so that the discovered groups correspond to meaningful and repeatable auxiliary operating patterns.

A second challenge is how to integrate the discovered auxiliary activity information into an energy forecasting model in a way that improves prediction performance without introducing error propagation or instability. In particular, the activity representation must be suitable for real-time use, robust across different vehicles and operating conditions, and compatible with standard forecasting models.

Therefore, the problem addressed in this thesis is to (1) discover and define auxiliary activity categories from unlabeled telemetry in a consistent and interpretable manner, and (2) quantify the benefit of using this inferred auxiliary activity as an additional input for energy consumption forecasting compared to a baseline model that does not use activity information.

1.3 RESEARCH QUESTIONS

The main research questions addressed in this thesis are as follows:

1. **How can auxiliary activity patterns be discovered from multivariate heavy-duty vehicle time series data without ground-truth labels?**
   This question focuses on building an unsupervised activity discovery pipeline, including signal selection, segmentation into time windows, and clustering choices and hyperparameters. The goal is to obtain stable clusters that represent distinct auxiliary activity patterns rather than noise.

2. **How can the discovered activity clusters be converted into usable activity labels for downstream modeling?**
   This question addresses how to summarize and interpret clusters, how to assign meaningful activity names with support from domain experts, and how to handle unclear or transition-like clusters. The goal is to create a practical and consistent auxiliary activity taxonomy.

3. **Does incorporating inferred auxiliary activity improve energy consumption forecasting compared to a baseline model without activity information, in offline and online settings?**
   This question evaluates the impact of auxiliary activity on forecasting by comparing a telemetry-only baseline with an activity-aware model that includes the inferred activity variable as input. Performance is evaluated both offline on a held-out test set and online using a prequential (test-then-train) protocol.

1.4 NOVELTY AND CONTRIBUTION

This thesis proposes a practical, reusable framework for auxiliary activity recognition and energy consumption forecasting in heavy-duty vehicles using real-world multivariate telemetry. Unlike approaches that assume pre-defined activity labels, this work addresses a realistic industrial setting in which auxiliary activity is not explicitly labeled and must be discovered from data.

The first contribution is an unsupervised, expert-in-the-loop activity discovery process for auxiliary systems. The approach clusters auxiliary-related signal patterns into recurring activity groups and, in collaboration with domain experts, assigns meaningful activity labels. This results in a usable auxiliary activity taxonomy that can be applied consistently for further modeling and analysis.

The second contribution is an end-to-end evaluation of activity-aware forecasting. The study compares a baseline model that uses

standard telemetry only with models that incorporate an inferred auxiliary activity variable (assigned using the clustering-derived taxonomy) as an additional input. The evaluation is conducted both offline on a held-out test set and online using a prequential test-then-train setup to assess whether activity-aware inputs improve prediction performance in both settings.

The third contribution is a modular pipeline architecture that operationalizes the proposed framework, from data preprocessing and activity discovery to activity labeling, feature generation, and forecasting evaluation. The pipeline is designed to be reusable and adaptable, enabling the same methodology to be applied to other vehicle equipment and to different classes of activities.

1.5 EVALUATION

1.5.1 *Activity Discovery Evaluation and Analysis (Segmentation and Clustering)*

Since auxiliary activity is discovered without ground-truth labels, the evaluation of activity recognition focuses on both quantitative and qualitative analysis. Quantitatively, segmentation methods are compared by measuring properties such as number of segments, segment length distributions, and sensitivity to parameter choices. Clustering quality is assessed using internal clustering metrics (e.g., silhouette score on feature-based clustering) and cluster stability across different time periods, vehicles, or sampling settings. Qualitatively, representative segments from each cluster are visualized and reviewed with domain experts to verify that clusters correspond to meaningful auxiliary operating patterns and to support the assignment of interpretable activity labels.

1.5.2 *Model Performance Evaluation (Energy Forecasting)*

The energy forecasting models are evaluated using standard regression metrics on a held-out test set. The primary metrics are Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), reported both overall and per forecast horizon when multi-step forecasts are used. In addition, training and validation loss curves are monitored to assess convergence and potential overfitting. The proposed activity-aware forecasting approach is compared against baseline forecasting methods that do not use auxiliary activity information, including simple statistical baselines and machine learning or deep learning models

1.5.3 *Online Learning Evaluation*

To study adaptation over time, an online learning setting is evaluated using a prequential (test-then-train) protocol. At each time step, the model first predicts future energy consumption and is then updated after the true values become available. Performance is summarized using prequential MAE over the stream and by comparing an offline-only model with an offline-plus-online adapted model. The effect of replay-buffer updates and trigger-based training activation is analyzed to understand when online updates provide measurable improvements.

2 LITERATURE REVIEW

Research on activity recognition and energy-related forecasting from sensor time series has expanded rapidly, particularly in settings where labeled data are scarce and operating conditions evolve over time. Self-supervised and contrastive learning methods have shown strong ability to learn useful representations from unlabeled multivariate signals. For example, Khaertdinov et al.[4] demonstrate that contrastive self-supervised learning can improve sensor-based activity recognition under limited labels, while Chang et al.[2] propose TimeDRL to learn disentangled representations for multivariate time series using predictive and contrastive pretext tasks, achieving strong results across forecasting and classification benchmarks. In transportation-focused work, Liu et al.[5] integrate distributional and spatio-temporal information to improve robustness for transportation activity recognition, highlighting the value of modeling temporal dependencies together with contextual structure.

In parallel, heavy-duty vehicle studies have applied unsupervised segmentation and clustering to obtain interpretable operational patterns and relate them to energy outcomes. Fang et al.[3] present an offline–online framework based on Toeplitz Inverse Covariance Clustering (TICC) to segment driving behaviors and connect discovered patterns to fuel consumption and contextual factors. For forecasting, Wang et al.[8] evaluate supervised machine learning approaches for predicting auxiliary energy consumption in heavy-duty battery electric vehicles, demonstrating that temporal models can capture operational dependencies but commonly assuming offline training. Overall, existing literature suggests that representation learning supports label-scarce recognition, clustering-based methods provide interpretable behavior discovery, and learning-based approaches can achieve accurate energy-related forecasts. However, fewer studies combine (i) label-efficient discovery of auxiliary activity patterns and (ii) their integration into forecasting pipelines under conditions that reflect real fleet operation, which motivates the focus of this thesis.

2.1 BACKGROUND

2.1.1 *Auxiliary-driven operational modes in vehicle telemetry*

Heavy-duty vehicle telemetry reflects the combined behavior of propulsion demand and multiple auxiliary subsystems. In practice, auxiliary

subsystems often operate in distinct modes or phases rather than as a single uniform load, and these modes can vary with ambient conditions and operational context. This motivates modeling auxiliary behavior as recurring operational patterns (activities) that can be extracted from sensor streams and used to provide additional context for downstream prediction tasks.

2.1.2 *Unsupervised activity discovery: segmentation and clustering of time series*

When explicit auxiliary activity labels are missing, a practical approach is to discover structure directly from data. A common workflow is to first identify relevant operating intervals (e.g., PTO-active cycles), then segment each interval into sub-sequences that represent changes in operating behavior, and finally cluster similar subsequences to obtain recurring activity patterns. The resulting clusters can be summarized and reviewed with domain experts to assign meaningful activity names, producing a usable auxiliary activity taxonomy that can be incorporated into forecasting models and further analysis.

2.1.3 *Time-series segmentation and change-point detection*

Segmentation is a common preprocessing step in time-series analysis when the goal is to identify distinct operating phases within longer sequences. Rather than assuming a single stationary pattern, segmentation methods aim to detect boundaries where the behavior of the signal changes, for example changes in mean level, variance, trend, or dependency structure. In vehicle telemetry, segmentation is particularly relevant because operational states can switch due to changes in load, environment, or control actions.

A widely used family of approaches is change-point detection, where the objective is to identify time indices that partition a sequence into approximately homogeneous segments according to a chosen cost function. Classical algorithms such as PELT are often used in practice due to their computational efficiency and their ability to control the number of detected change points through a penalty term. In such methods, the selected cost model (e.g., an l_2 cost targeting changes in mean) and the penalty parameter directly affect the resulting segmentation granularity, which can have a strong impact on downstream analysis such as clustering and forecasting.

More recent work has focused on unsupervised change-point detection for multivariate time series in settings where the number of change points is unknown and extensive parameter tuning is undesir-

able. For example, Wu et al. [9] propose an unsupervised method that clusters distribution estimates from sliding windows and applies an MDL-guided greedy selection procedure to infer meaningful change points in a scalable manner. Such approaches highlight that practical change-point detection in real telemetry often requires methods that are robust to varying operating conditions and that can capture changes beyond simple univariate mean shifts.

2.1.4 *Time-series clustering and similarity measures*

After segmentation, the resulting sub-sequences must be grouped into recurring patterns. A common approach is feature-based clustering, where each segment is summarized by engineered statistics (e.g., mean, standard deviation, extrema, energy, duration, and slope features) and then clustered using algorithms such as KMeans. Feature-based clustering is simple and efficient, but its effectiveness depends on the choice of features and scaling.

An alternative is shape-based clustering, which groups segments using distances computed directly on the time-series shape. Dynamic Time Warping (DTW) is a widely used similarity measure for sequences that may be misaligned in time, as it allows non-linear alignment between sequences. In practical settings, segments often have variable length, which motivates either resampling segments to a common length before clustering, or using methods that can compare variable-length sequences directly. DTW-based clustering is therefore particularly relevant when the temporal profile of the segment carries important information beyond simple summary statistics.

Recent work has improved DTW-based clustering for multivariate time series by addressing unequal contributions of different dimensions during alignment and distance computation. For example, Cai et al. [1] propose a data-adaptive DTW-based fuzzy clustering method that introduces locally weighted DTW to regulate dimension importance in an unsupervised setting.

2.2 MULTI-STEP FORECASTING FOR ENERGY-RELATED SIGNALS

Energy-related forecasting in vehicle applications is typically formulated as a multi-step prediction problem, where the goal is to predict a sequence of future values over a horizon rather than a single time step. Multi-step forecasting can be approached in different ways, including recursive strategies (iteratively predicting one step ahead) and direct strategies (predicting the full horizon in one model output). Direct multi-horizon prediction is often preferred when error accumulation in recursive methods is a concern.

Forecasting performance is commonly evaluated using error metrics such as MAE and RMSE, reported either as aggregate values over the entire horizon or as horizon-dependent errors (e.g., short-term versus long-term). Recent work on multi-horizon forecasting emphasizes that different model classes may perform best at different horizons, motivating horizon-aware evaluation rather than relying only on a single-step metric [7]. In addition to average errors, qualitative evaluation on representative operating cycles is often used to understand failure modes, such as regime transitions or abrupt load changes.

2.3 ONLINE AND CONTINUAL LEARNING FOR STREAMING TIME SERIES

In real-world deployments, vehicle telemetry arrives as a stream and operating conditions can change over time due to factors such as weather, route characteristics, payload, driver behavior, and system aging. Models trained offline on historical data may therefore degrade when the underlying data distribution shifts. Online and continual learning address this challenge by updating model parameters during operation using newly observed data, aiming to maintain performance over time without requiring full retraining from scratch.

A common evaluation protocol for online learning in forecasting is the prequential (test-then-train) setting, where the model first produces a prediction for the next time step or horizon and is then updated once the true outcome becomes available. Practical online learning systems often combine periodic update schedules with replay mechanisms to avoid catastrophic forgetting, for example by storing a buffer of recent examples and sampling minibatches for incremental updates. Other strategies include trigger-based adaptation, where updates are activated only when an error signal exceeds a threshold, focusing learning effort on periods when the model is underperforming. These ideas are relevant to heavy-duty vehicle energy forecasting because they provide a structured way to adapt forecasting models over time while keeping training costs manageable.

3 METHOD

This section describes the methodology used to recognize auxiliary activity patterns and forecast energy consumption from heavy-duty vehicle telemetry. The overall approach is organized as a pipeline that processes raw time-indexed signals, trains forecasting models in an offline setting, and adapts the models online using a prequential (test-then-train) protocol. The following sections detail the data preparation steps, the activity discovery procedure, the forecasting model architectures, and the loss functions and update strategy used for both offline training and online adaptation.

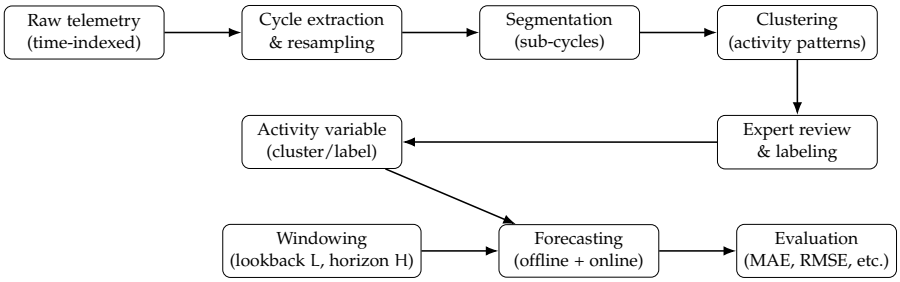

Figure 1: Overview of the proposed pipeline for auxiliary activity discovery and forecasting.

3.1 MODEL ARCHITECTURE

The proposed approach follows a pipeline architecture that connects auxiliary-related signal processing with multi-step forecasting. The workflow starts from raw vehicle telemetry and proceeds through (i) cycle extraction and resampling, (ii) window generation for supervised learning, (iii) offline forecasting using multiple model families, and (iv) online adaptation using a prequential learning protocol. This design ensures that training instances are constructed consistently and do not cross cycle boundaries, while enabling a fair comparison across different forecasting architectures.

3.1.1 *Cycle Extraction and Resampling*

Telemetry data are first filtered per vehicle and sorted by time. Operation cycles are identified by splitting the continuous time series whenever the time gap between consecutive measurements exceeds

a predefined threshold. Formally, a new cycle starts at index t whenever the inter-sample time difference satisfies:

$$\Delta t_t = \text{time}(t) - \text{time}(t-1) > \tau_{\text{gap}}$$

where τ_{gap} is a tunable gap threshold. To focus on active operation, cycles are filtered by retaining only those where at least a fraction ρ of samples have positive power. For a cycle c with samples $\{p_{c,1}, \ldots, p_{c,n_c}\}$, the positive fraction is:

$$r_c = \frac{1}{n_c} \sum_{i=1}^{n_c} \mathbb{I}\left[p_{c,i} > 0\right]$$

and the cycle is retained if:

$$r_c \geqslant \rho.$$

Each retained cycle is then resampled to a uniform time grid, and missing values are handled through interpolation within the cycle.

3.1.2 *Activity Discovery (Segmentation and Clustering)*

Since explicit auxiliary activity labels are not available, auxiliary activity is modeled as a pattern discovery problem using unsupervised learning. The goal is to identify recurring operating patterns from auxiliary-related signals and represent them as an auxiliary activity variable that can later be used for analysis and forecasting.

Each valid cycle is divided into smaller sub-cycles that capture changes in operating behavior within the same cycle. In this work, segmentation is performed using change-point detection (PELT with an l_2 cost model and a penalty parameter) to identify boundaries where the statistical behavior of the power signal changes. Additional segmentation strategies, such as matrix-profile-based boundary detection and energy-based segmentation, are also explored to understand how different criteria affect the resulting sub-cycles. The output of this step is a set of variable-length sub-cycles, each defined by its start and end indices within the original cycle.

To enable clustering, each sub-cycle is represented either (i) by engineered statistical features or (ii) by a shape representation. The engineered feature set includes quantities such as segment duration, mean power, standard deviation, extrema, energy, and slope-related statistics derived from first differences. For shape-based clustering, sub-cycles are resampled to a common length when required by the clustering algorithm.

Unsupervised clustering is applied to group similar sub-cycles into recurring activity patterns. Feature-based clustering is performed using KMeans on standardized sub-cycle feature vectors. Shape-based

clustering is performed using DTW-based `TimeSeriesKMeans` on resampled sub-cycles to capture similarities in temporal profiles. In addition, hybrid strategies are explored, where obvious regimes (e.g., low-power segments or predefined power bands) are assigned using rule-based thresholds and the remaining segments are clustered using KMeans. The resulting cluster IDs form an initial activity taxonomy. Finally, the discovered clusters are reviewed together with domain experts to assign interpretable auxiliary activity labels and to merge or refine clusters that represent mixed behavior or transitions.

3.1.3 *Fixed-Length Windowing*

After cycle preparation, each cycle is transformed into supervised learning instances using a sliding-window formulation. For each instance, the input is a lookback window of length L and the output is a forecasting horizon of length H. For a univariate target sequence $\{y_t\}$ (here, y_t denotes the resampled target power signal), each training instance at time index t is:

$$\mathbf{x}_t = [y_{t-L}, \ldots, y_{t-1}] \in \mathbb{R}^L$$

$$\mathbf{y}_t = [y_t, \ldots, y_{t+H-1}] \in \mathbb{R}^H$$

Windows are generated only within each cycle, ensuring that no instance spans across two cycles. This produces an instance stream used for offline model training and testing, and it also supports online learning experiments where windows arrive sequentially in time.

3.1.4 *Forecasting Models*

Multiple forecasting models are implemented to establish baselines and evaluate model capacity for multi-step prediction. A simple statistical baseline predicts the mean of the lookback window repeated over the full horizon:

$$\hat{y}_{t+h} = \frac{1}{L} \sum_{j=1}^{L} y_{t-j} \qquad \text{for } h = 0, \ldots, H-1$$

For deep learning, an LSTM-based model is used to map the input sequence to a horizon-length output through a sequence encoder followed by fully connected layers:

$$\hat{\mathbf{y}}_t = f_\theta(\mathbf{x}_t)$$

To better capture long-range dependencies, an LSTM with self-attention is also implemented. Given LSTM hidden states $\mathbf{H}_t \in \mathbb{R}^{L \times d}$ for one instance, dot-product self-attention is computed as:

$$\mathbf{A}_t = \text{softmax}\left(\frac{\mathbf{H}_t \mathbf{H}_t^\top}{\sqrt{d}}\right)$$

where softmax is applied row-wise so that each row of $\mathbf{A}_t$ sums to 1.

$$\mathbf{Z}_t = \mathbf{A}_t \mathbf{H}_t$$

followed by temporal pooling (global average pooling) to obtain a fixed-length representation:

$$\mathbf{z}_t = \frac{1}{L} \sum_{i=1}^{L} \mathbf{Z}_t[i,:]$$

which is passed to a regression head to predict $\hat{\mathbf{y}}_t$. Finally, a multi-layer perceptron (MLP) forecaster is used as an additional lightweight model for comparison and for efficient repeated updates in the online learning setting.

3.1.5 *Online Learning Architecture (Prequential Adaptation)*

To evaluate adaptation over time, an online learning setup is implemented following a prequential (test-then-train) protocol. At each incoming instance t, the model first produces a multi-step prediction:

$$\hat{\mathbf{y}}_t = f_{\theta_t}(\mathbf{x}_t)$$

Once the ground-truth horizon $\mathbf{y}_t$ is available, the forecasting loss is computed and the labeled instance $(\mathbf{x}_t, \mathbf{y}_t)$ is stored in a replay buffer that maintains the most recent Q instances. Model updates are performed periodically: every N instances the model is trained for P minibatch iterations by sampling from the replay buffer. To focus updates on periods with degraded performance, a trigger mechanism is optionally applied. Using the instance-level MAE:

$$\ell_t = \frac{1}{H} \sum_{h=0}^{H-1} |y_{t+h} - \hat{y}_{t+h}|$$

training is activated for the next K instances if ℓ_t exceeds a predefined threshold ε. In addition, a bounded learning-rate schedule is used during streaming updates:

$$\eta(t) \in [\eta_{\min}, \eta_{\max}].$$

3.2 LOSS FUNCTIONS

Forecasting models are optimized using regression loss functions. In this work, Mean Absolute Error (MAE) is used as the primary training objective for neural forecasting models:

$$\mathcal{L}_{\text{MAE}}(\theta) = \frac{1}{B} \sum_{t=1}^{B} \frac{1}{H} \sum_{h=0}^{H-1} |y_{t+h} - \hat{y}_{t+h}|$$

where B denotes the batch size. For reporting and comparison, Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 are computed on the test set:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$$

$$\bar{y} = \frac{1}{N}\sum_{i=1}^{N} y_i$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}$$

In the online learning setup, the same forecasting loss is used during incremental updates on minibatches sampled from the replay buffer under the prequential protocol.

4 DATASET DESCRIPTION

4.0.1 *Dataset*

This thesis uses an volvo dataset consisting of time-indexed heavy-duty vehicle telemetry collected during real operation. The dataset contains multivariate measurements related to vehicle usage and auxiliary operation. In the experiments, the analysis focuses on a target auxiliary-related power signal, denoted as $\{p_t\}$, which is used for both auxiliary activity pattern discovery and multi-step forecasting. The experiments are conducted on data from multiple vehicles, which are referenced using anonymized identifiers.

The raw telemetry contains gaps and is not uniformly sampled. Therefore, the data is reorganized into operating cycles that represent continuous usage intervals of the target auxiliary function. This cycle-based representation serves as the basis for subsequent segmentation, clustering, and forecasting experiments.

4.0.2 *Data Preprocessing*

First, the data is filtered by vehicle and sorted by time. Cycles are identified by splitting the continuous stream whenever the time gap between consecutive samples exceeds a threshold τ_{gap}. To focus on active operation, each cycle is retained only if at least a fraction ρ of its samples have strictly positive target power.

Each retained cycle is resampled independently to a uniform time grid, and missing values are handled by interpolation within the cycle to avoid mixing information across cycle boundaries. The resulting cycles form the basis for both activity discovery and forecasting experiments.

For offline forecasting experiments, cycles are split into training and test sets at the cycle level using a fixed split ratio. For online learning experiments, data are ordered chronologically per vehicle and split into rolling evaluation blocks, where the model is trained on an earlier block and then evaluated and updated sequentially on a later block using a prequential protocol.

5 EXPERIMENTAL EVALUATION

In this section, we evaluate the proposed framework on an anonymized operational telemetry dataset collected from Volvo vehicles during real-world operation. The evaluation is conducted in two dimensions: (i) unsupervised auxiliary activity discovery from extracted operating cycles, and (ii) multi-step forecasting of the target power signal, including offline baselines and online adaptation.

5.1 TRAINING SETUP

5.1.1 *Data and Task Definition*

All experiments use an auxiliary-related target power signal and operate on cycles extracted from continuous telemetry using a time-gap criterion. Cycles are filtered to retain sustained operation using a minimum positive-power fraction, and each retained cycle is resampled to a uniform time grid. Forecasting is formulated as direct multi-step prediction with lookback length L and horizon H. Sliding windows are generated only within each cycle to avoid crossing cycle boundaries.

Auxiliary activity recognition is evaluated as a segmentation-and-clustering pipeline. Cycles are divided into sub-cycles using change-point detection (PELT with an l_2 cost model and a penalty parameter). Each sub-cycle is represented using engineered statistical features (e.g., mean, variability, extrema, trend, duration, and energy) and, where applicable, a shape representation. Clustering is performed using feature-based KMeans on standardized feature vectors, and shape-based clustering is explored using DTW-based `TimeSeriesKMeans` after resampling sub-cycles to a common length. Since ground-truth activity labels are not available, results are analyzed using quantitative summaries (e.g., cluster size distributions) and qualitative visualizations, followed by domain expert review for interpretation.

Offline forecasting compares multiple model families using a cycle-level split. The evaluated models include a mean baseline and neural sequence models (e.g., LSTM-based forecasters with and without an attention mechanism). Neural models are trained using the Adam optimizer with an MAE loss and a validation split for model selection.

To reflect time-ordered deployment, rolling blocks are used where the model is trained on an earlier interval and then evaluated on a subsequent interval. Online learning is evaluated using test-then-

update protocols, including (i) instance-level prequential learning with replay-based updates and a trigger mechanism, and (ii) cycle-wise updates where the model is updated after each test cycle. Performance is summarized per rolling block and compared against an offline-only baseline trained on the same block.

6 RESULTS

6.1 PTO CYCLE EXTRACTION RESULTS

PTO cycles were extracted by grouping consecutive PTO-active samples into segments, where a new cycle starts whenever the time gap between two consecutive PTO-active samples exceeds a threshold τ. Extracted cycles were then filtered to retain intervals with sustained PTO operation by requiring that at least a fraction ρ of samples within each cycle exhibit strictly positive PTO power.

6.1.1 *Examples of Retained and Excluded PTO Cycles*

To illustrate the cycle filtering criterion, we show one retained (valid) cycle and one excluded (invalid) cycle. A cycle is retained if at least 90% of its samples satisfy `pto_power_kw` > 0.

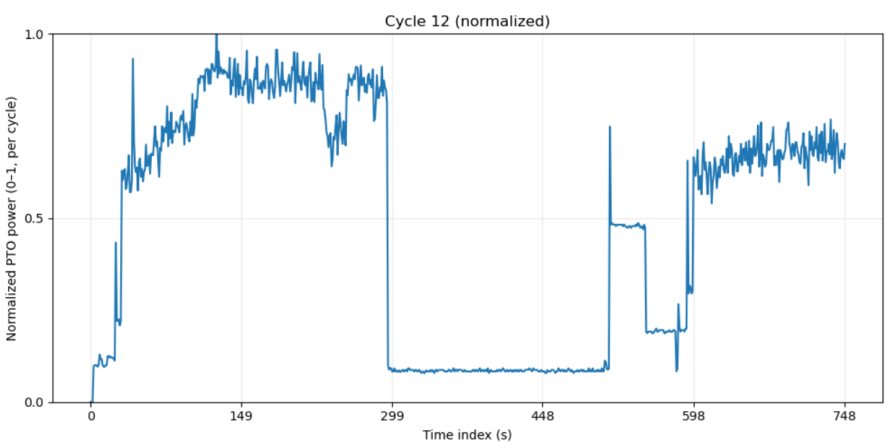

Figure 2: Valid cycle

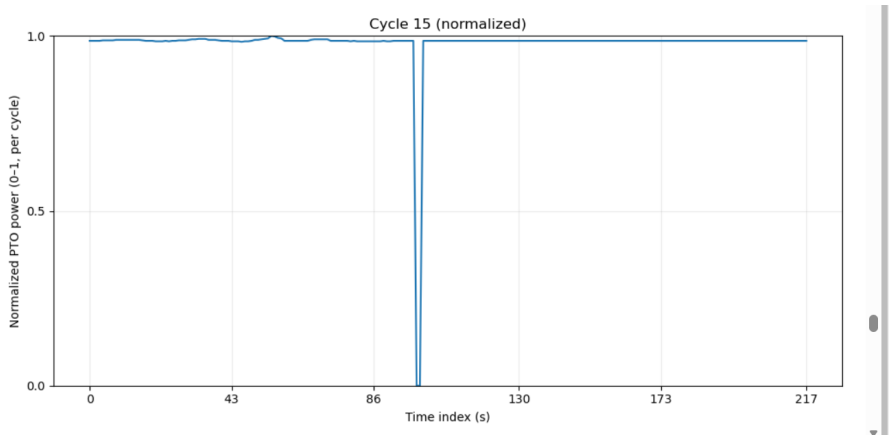

Figure 3: Invalid cycle

6.1.2 *Retained (valid) cycle.*

A retained PTO cycle contains predominantly positive PTO power values, indicating continuous PTO activity and making it suitable for downstream sub-cycle segmentation and clustering.

6.1.3 *Excluded (invalid) cycle.*

An excluded cycle contains a large fraction of non-positive PTO power values, which typically indicates inactive PTO operation or fragmented signals. Such cycles are removed to avoid introducing non-operational patterns into segmentation and clustering.

6.2 SUB-CYCLE SEGMENTATION RESULTS

After extracting valid PTO cycles, each cycle was further segmented into sub-cycles to capture changes in operating behavior within the same PTO-active interval. Multiple segmentation strategies were explored during development (e.g., rule-based segmentation, change-point detection, and spectrogram-based segmentation). This subsection reports the results for derivative-based segmentation, which detects sub-cycle boundaries based on large changes in PTO power.

6.2.1 *Derivative-based segmentation.*

For each valid PTO cycle, the PTO power signal was smoothed using a rolling mean with window length w to reduce noise. Sub-cycle boundaries were then detected by thresholding the magnitude of the estimated time-derivative, i.e., a boundary was introduced whenever

$$\left| \frac{dP_{PTO}(t)}{dt} \right| > \delta,$$

where δ is a tunable change-point threshold. Finally, segments shorter than a minimum length L_{min} were discarded to avoid spurious sub-cycles. Using this configuration, a total of **[N_deriv]** sub-cycles were extracted from the valid PTO cycles in the dataset.

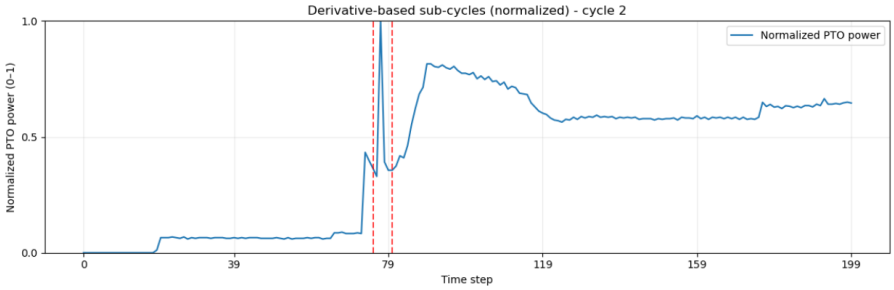

Figure 4: derivative based

Overall, the derivative-based method produces sub-cycles that align with rapid increases or decreases in PTO power, which is useful when operating phases are characterized by step-like transitions. The number and location of detected boundaries depend on the selected derivative threshold and the smoothing configuration, which can affect the granularity of the resulting sub-cycles. The extracted sub-cycles are used as inputs for downstream clustering to discover recurring auxiliary activity patterns.

6.2.2 *Matrix Profile*

Each cycle was segmented into sub-cycles using a matrix-profile-based approach. For each cycle, the matrix profile was computed using a subsequence length m. Candidate boundaries between sub-cycles were identified by selecting prominent peaks in the matrix profile, which correspond to locations of high dissimilarity between neighboring subsequences. To prevent selecting boundaries that are too close to each other, an exclusion zone proportional to the subsequence length was applied around each selected peak. Finally, the number of sub-cycles was controlled by specifying a target number of segments n_{seg}, yielding approximately $n_{seg} - 1$ internal boundaries per cycle.

Figure 5 shows an example PTO cycle with the detected sub-cycle boundaries. The boundaries typically appear at time regions where the local signal pattern differs most from other parts of the cycle, indicating potential transitions between different PTO operating phases. These segmented sub-cycles are used as the input for downstream clustering to discover recurring auxiliary activity patterns.

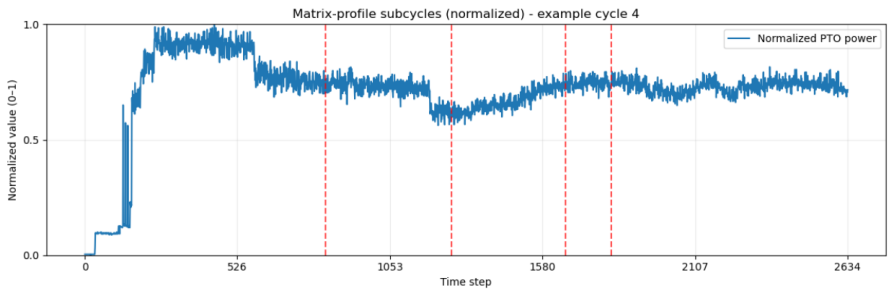

Figure 5: Matrix profile

6.2.3 *Change-Point Detection*

Sub-cycle segmentation was performed using change-point detection to identify transitions between distinct operating phases within each cycle. Change points were detected using the PELT algorithm with an l_2 cost model, which identifies boundaries corresponding to shifts in the mean level of the signal. A penalty term λ was used to control the trade-off between sensitivity and the number of detected segments. The output of the algorithm is a set of breakpoints (segment end indices) that define the resulting sub-cycles within each cycle. Figure 6 illustrates an example cycle with the detected change points visualized as vertical dashed lines.

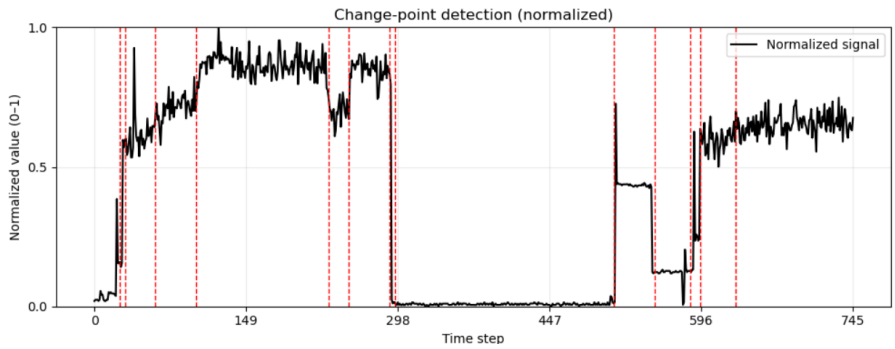

Figure 6: Change point detection

Qualitative inspection indicates that detected change points typically occur at locations where the PTO power signal shifts between distinct operating levels. The resulting sub-cycles are used as inputs for downstream clustering to discover recurring auxiliary activity patterns.

6.2.4 *Window Correlation*

Sub-cycle segmentation was performed using a window-correlation method. The signal was scanned in time order and split into sub-cycles by comparing two consecutive windows of equal length. The window length was set as a fraction of the cycle length, with a min-

imum size w_{min}. A boundary was added when the Pearson correlation between the two windows fell below a threshold γ. To avoid over-segmentation, boundaries were only allowed if they were at least L_{min} samples apart, and very short segments were merged with a neighboring segment.

Figure 7 shows an example PTO cycle segmented using the window-correlation method. The detected boundaries typically appear at locations where the PTO power pattern changes relative to the previous behavior, producing sub-cycles that are more homogeneous in shape. These sub-cycles are used as inputs for downstream clustering to discover recurring auxiliary activity patterns.

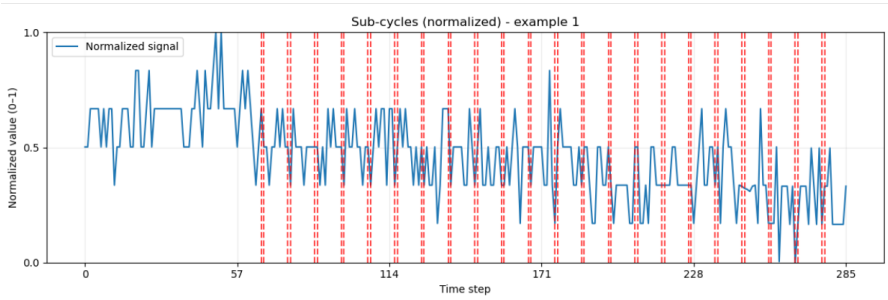

Figure 7: Window based

6.2.5 *Energy-Based*

Sub-cycle segmentation was evaluated using an energy-based criterion derived from the PTO power signal. For each cycle, cumulative energy was computed by integrating PTO power over time. Let p_t denote PTO power and Δt_t the time difference between consecutive samples. The cumulative energy is computed as:

$$E_t = \sum_{i=1}^{t-1} p_i \cdot \Delta t_i.$$

A new sub-cycle boundary is introduced whenever the increase in cumulative energy since the previous boundary exceeds a threshold ΔE_{min}, subject to a minimum separation of L_{min} samples between consecutive boundaries. The resulting sub-cycles are then used as inputs for downstream clustering to discover recurring auxiliary activity patterns.

Figure 8 shows an example PTO cycle segmented using the energy-based method, where red dashed lines indicate detected sub-cycle boundaries.

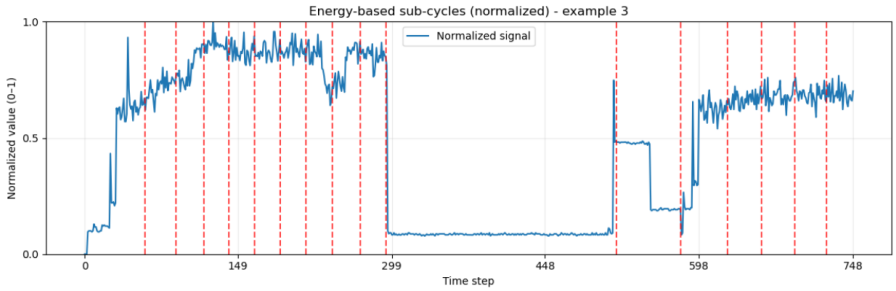

Figure 8: Energy based

6.3 SUB-CYCLE CLUSTERING RESULTS

This section reports the results of sub-cycle clustering, aiming to discover recurring operating patterns from segmented cycles. Sub-cycles are grouped using multiple unsupervised approaches, including feature-based KMeans, DTW-based shape clustering (after resampling), and a hybrid rule-based strategy that separates clear regimes before clustering the remaining segments. In addition, spectrogram-based and step-level clustering are explored as alternative views of the signal behavior. Cluster assignments are visualized by overlaying colored segments on power profiles, and the resulting clusters form candidate activity categories for domain expert labeling.

6.3.1 *PELT + Feature-based KMeans*

Sub-cycles were clustered to discover recurring operating patterns. First, each cycle was segmented into sub-cycles using change-point detection with the PELT algorithm (with an l_2 cost model) and a penalty parameter λ controlling the number of detected change points. For each resulting sub-cycle, a feature vector was extracted to capture both operating level and temporal shape characteristics. The extracted features include segment length, summary statistics (e.g., mean, extrema, standard deviation), energy-related measures, slope-related statistics, and start and end values, computed both in the original scale and in a normalized form.

All sub-cycle feature vectors were standardized (z-score normalization) and clustered using KMeans with a chosen number of clusters k. Each sub-cycle was assigned a cluster ID, and the assignments were mapped back to the original cycles for visualization.

Figure 9 illustrates an example cycle with colored sub-cycle regions corresponding to the assigned cluster IDs. The discovered clusters represent recurring sub-cycle patterns and serve as the basis for subsequent expert interpretation and activity labeling.

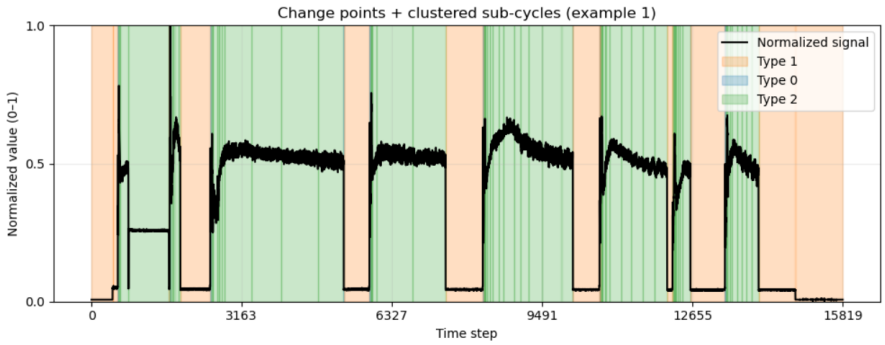

Figure 9: change point and K means

6.3.2 *Change-Point Segmentation + DTW-based Clustering*

Sub-cycles were obtained by segmenting each cycle using PELT change-point detection (with an l_2 cost model and penalty parameter λ). The resulting sub-cycles were then clustered by temporal shape using DTW-based TimeSeriesKMeans.

Because the extracted sub-cycles have variable lengths, each sub-cycle was resampled to a common target length T_{target} prior to clustering. The resampled sub-cycles were clustered using TimeSeriesKMeans with Dynamic Time Warping (DTW) distance, using a chosen number of clusters k and standard optimization settings (e.g., maximum iterations and multiple initializations).

Each sub-cycle was assigned a DTW cluster label, and the cluster assignments were mapped back to the original cycles for visualization. Figure 10 illustrates an example where sub-cycles are highlighted with colors corresponding to their DTW cluster IDs. The resulting DTW clusters capture recurring sub-cycle shapes and are used for expert interpretation and subsequent activity labeling.

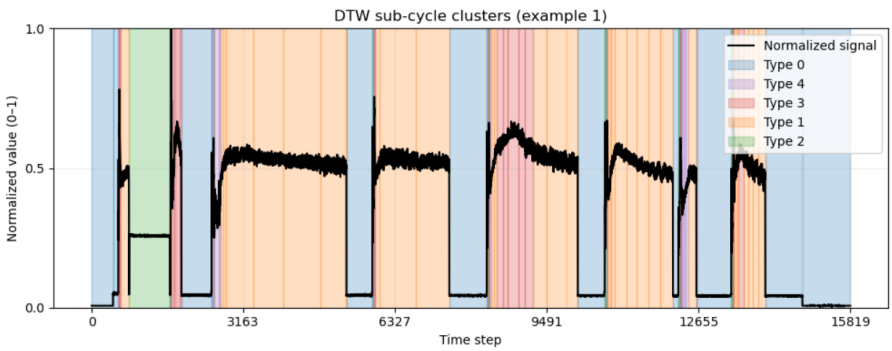

Figure 10: change point and DTW

6.3.3 *Hybrid Rule-based + KMeans*

Sub-cycles were clustered using a hybrid approach that combines rule-based grouping with feature-based clustering. First, sub-cycles representing near-idle operation were assigned to a dedicated low-power cluster. Second, sub-cycles whose mean power fell within a predefined operating band were assigned to a reserved cluster capturing this regime. The remaining sub-cycles were represented using engineered statistical features (e.g., mean, standard deviation, extrema, percentiles, trend, fluctuation, and duration) and standardized using z-score normalization. KMeans clustering was then applied to the standardized feature vectors using a configured number of clusters k, and the resulting labels were mapped to a consistent cluster ID scheme.

Figure 11 illustrates an example cycle where the cluster assignments are mapped back to the time axis. The rule-based clusters and the additional KMeans-derived clusters highlight recurring sub-cycle regimes and are used for downstream interpretation and activity labeling.

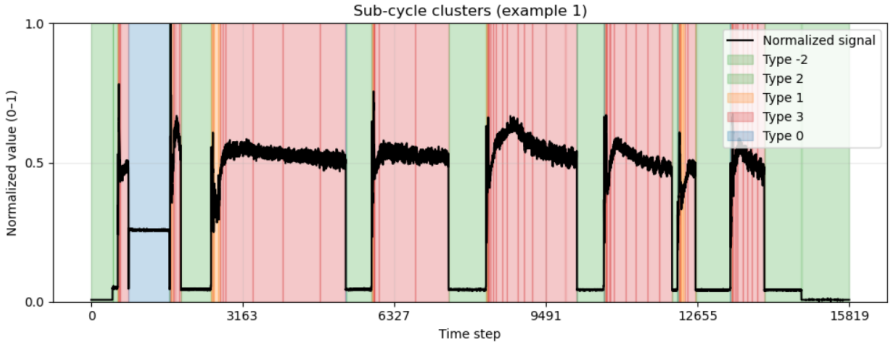

Figure 11: Hybrid rule based + Kmeans

6.3.4 *Spectrogram-Based Clustering*

In the spectrogram-based approach, clustering is performed at the STFT frame level. Each frame is represented by its log-magnitude spectrum and assigned a cluster label using KMeans. When mapping frame-level labels back to the time-domain signal, some samples may remain unassigned in the visualization. This can occur when short runs of labels are removed by a minimum-duration constraint, so rapid label switching may lead to gaps after filtering. In addition, STFT analysis represents the signal using overlapping frames with a discrete hop size, so converting frame labels back to sample-level regions is inherently approximate and can leave small unmatched regions, especially near cycle boundaries.

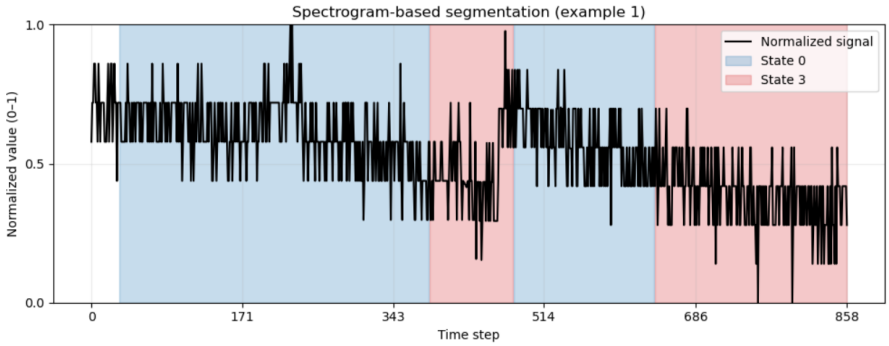

Figure 12: Spectrum based

6.3.5 *Step-level KMeans on Fixed-Length Cycle)*

In this experiment, clustering was performed at the time-step level to identify recurring local patterns within cycles. Each valid cycle was resampled to a fixed length T_{fixed} using interpolation, which enables direct comparison across cycles of different duration.

For each resampled cycle $\{p_t\}_{t=1}^{T_{\text{fixed}}}$, a feature vector was constructed at every time step using the signal value and its first and second discrete differences:

$$\mathbf{x}_t = \left[p_t, \Delta p_t, \Delta^2 p_t\right],$$

where:

$$\Delta p_t = p_t - p_{t-1},$$
$$\Delta^2 p_t = \Delta p_t - \Delta p_{t-1}.$$

All step-level feature vectors were standardized using z-score normalization and clustered using KMeans with a chosen number of clusters K_{step}. The resulting step labels were mapped back to each cycle and converted into contiguous segments by merging consecutive time steps with the same cluster label. Figure 13 illustrates an example cycle with the resulting step-level segments overlaid on the power signal.

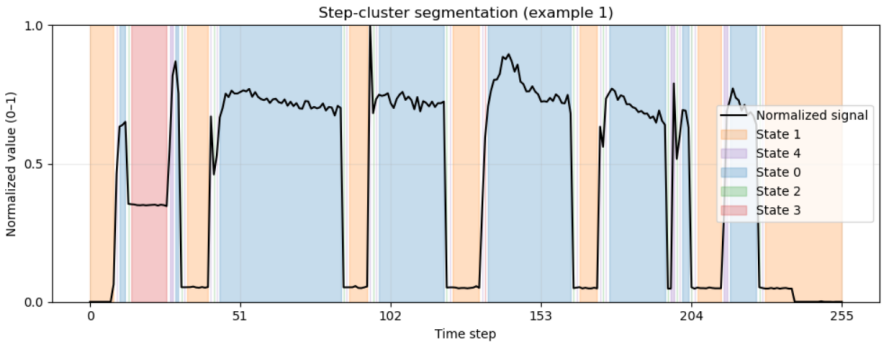

Figure 13: Step level Kmeans on fixed length

6.4 FORECASTING RESULTS (REGRESSION)

This section reports regression results for multi-step forecasting of the PTO power signal. We first evaluate offline forecasting performance using a cycle-level train/test split and compare a statistical baseline with neural sequence models. We then assess time-ordered performance using rolling blocks and compare offline-only training against two online adaptation strategies: cycle-wise test-then-update and prequential replay-buffer updates.

6.4.1 *Offline Multi-step Forecasting Setup*

The regression task is formulated as direct multi-step forecasting of the target power signal using resampled cycles. For each cycle, supervised learning instances are generated using a sliding-window approach with lookback length L and forecasting horizon H. Windows are generated within cycles only to avoid crossing cycle boundaries. The data is split at the cycle level into training and test sets using a fixed split ratio, and results are reported on the held-out test set.

6.4.1.1 *Baselines and Models*

A mean baseline is used as a reference model by predicting the average value of the input window and repeating it over the full forecasting horizon. In addition, neural sequence models are evaluated, including an LSTM-based forecaster and an LSTM augmented with a self-attention mechanism. In the attention-based model, dot-product self-attention is applied over the LSTM hidden states, followed by a pooling operation and a regression head to produce the multi-step forecast. The neural models are trained using the Adam optimizer with an MAE loss, using a validation split for model selection..

6.4.1.2 *Qualitative Forecast Visualization*

To complement the aggregate error metrics, a qualitative comparison of the forecasting models is shown for an example cycle. The plot shows the ground-truth power signal together with the predicted horizons from the mean baseline, the LSTM model, and the LSTM with attention.

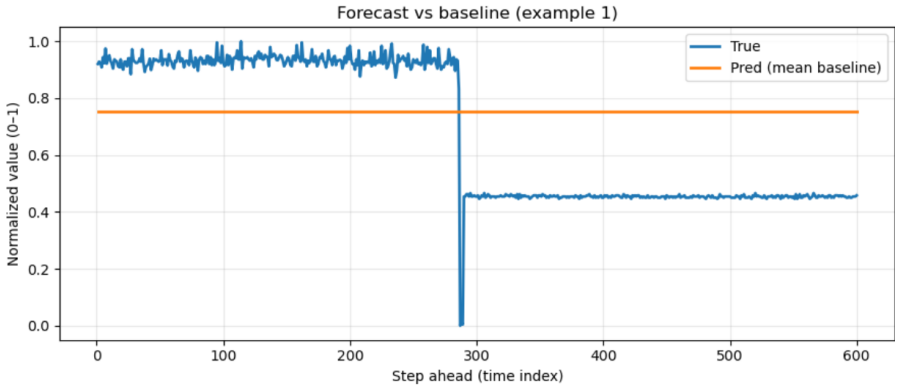

Figure 14: Mean based

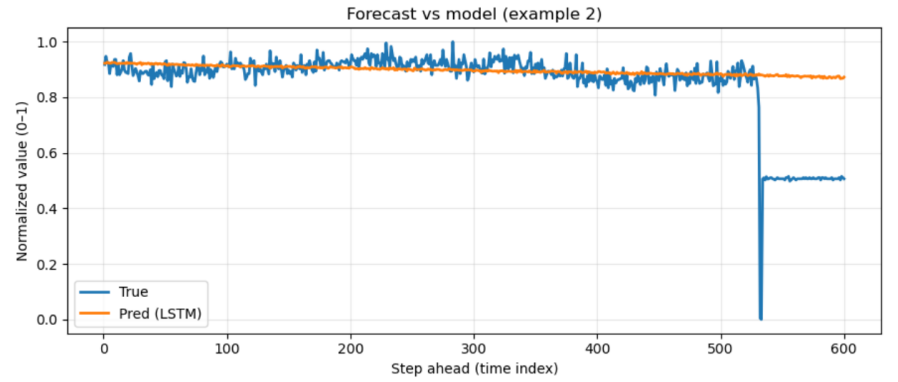

Figure 15: LSTM

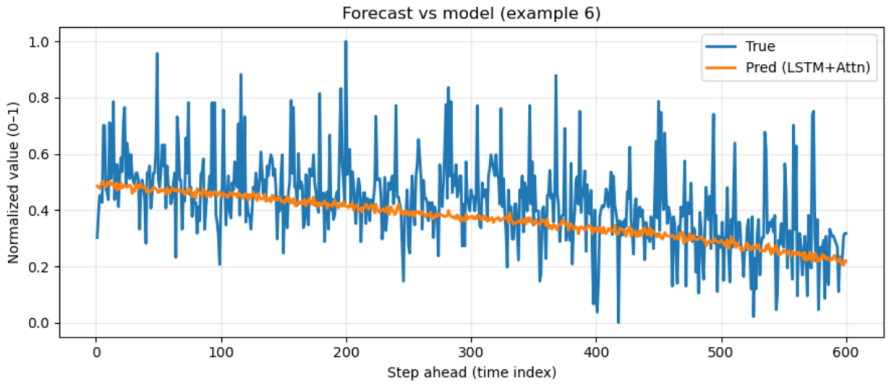

Figure 16: LSTM and attenuation

Table 1: Offline forecasting performance on the test set (normalized error metrics).

Model	$NMAE_{range}$	$NRMSE_{range}$	$NMSE_{var}$
Mean baseline	0.056747	0.108944	0.655868
LSTM	0.033102	0.093620	0.484032
LSTM + Attention	0.034479	0.091166	0.459484

Table 1 shows that both neural forecasting models outperform the mean baseline, indicating that the temporal structure in the power signal provides useful predictive information beyond a constant average forecast. The LSTM and the attention-augmented LSTM achieve broadly comparable performance, with only modest differences across metrics in this configuration. Overall, these results support the use of sequence models for multi-step forecasting relative to a simple statistical baseline.

6.5 ONLINE LEARNING RESULTS

Online learning is evaluated using time-ordered rolling blocks, where the model is first trained offline on an earlier period and then updated during a later test period. Two online strategies are considered: a cycle-wise test-then-update approach and an instance-level prequential approach with a replay buffer and trigger mechanism. Performance is compared against an offline-only baseline using MAE and MSE over the full multi-step horizon to quantify the benefit of online adaptation.

6.5.1 *LSTM Cycle-wise Test-then-Update*

Online learning was evaluated using a cycle-wise test-then-update protocol on an anonymized vehicle dataset. Cycles were extracted us-

ing a time-gap threshold τ and filtered to retain sustained operation by requiring that at least a fraction ρ of samples within a cycle exhibit positive power. Each retained cycle was resampled to a fixed time grid, and forecasting was formulated as direct multi-step prediction of the target power signal with lookback length L and forecasting horizon H. Windows were generated within cycles only to avoid crossing cycle boundaries.

A rolling-block evaluation was performed, where an initial interval was used for offline training followed by a subsequent interval for testing. For each rolling block, an LSTM model was first trained offline on the training interval using the Adam optimizer with learning rate η_{off}, an MAE loss, and fixed training hyperparameters (e.g., number of epochs and batch size). During the test interval, cycles were processed in chronological order and evaluated in a test-first manner.

For each test cycle, the model first produced predictions for all windows in the cycle and then performed a small number of gradient updates on that cycle's windows using a reduced online learning rate η_{on}. Performance was summarized per rolling block using MAE and MSE computed over all predicted values in the test interval. In addition to aggregate metrics, qualitative plots of the true versus predicted power signal on representative cycles were used to assess how well the model tracks changes over time.

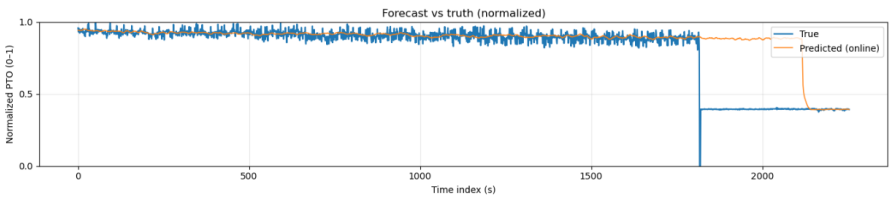

Figure 17: LSTM online

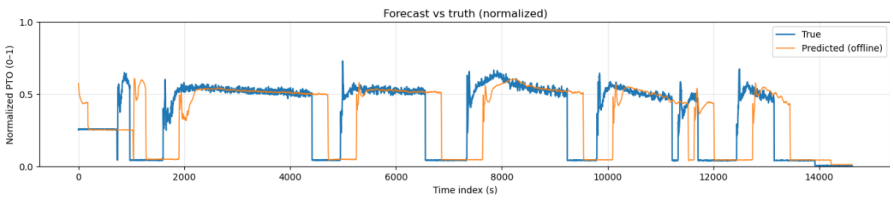

Figure 18: LSTM offline

Online cycle-wise updates achieve equal or lower error than the offline baseline in all evaluated blocks.

6.5.2 *MLP-Based Prequential Learning with Replay Buffer*

Online learning was evaluated using a prequential (test-then-train) protocol at the window (instance) level. Forecasting is formulated as direct multi-step prediction of a target power signal using sliding windows with lookback length L and forecasting horizon H. Windows are generated within each cycle only to avoid crossing cycle boundaries.

In this setting, a lightweight MLP forecaster is used due to its low computational cost and suitability for frequent incremental updates. The model maps a flattened lookback window of length L to a horizon-length output of size H using a feed-forward network. For each rolling block, the model is first trained offline on an initial interval and then evaluated on a subsequent interval in chronological order using the prequential protocol.

During the evaluation interval, each instance is stored in a replay buffer containing the most recent Q instances. Model updates are performed periodically (e.g., every N instances) for a fixed number of minibatch iterations P. A trigger mechanism is used to focus adaptation: if the instance-level error exceeds a threshold ε, additional training is activated for the next K instances. A bounded learning-rate schedule is applied during streaming updates, with learning rate constrained to $[\eta_{min}, \eta_{max}]$. Performance is compared against an offline-only rolling baseline and reported using MAE and MSE computed over all predicted values in each evaluation interval.

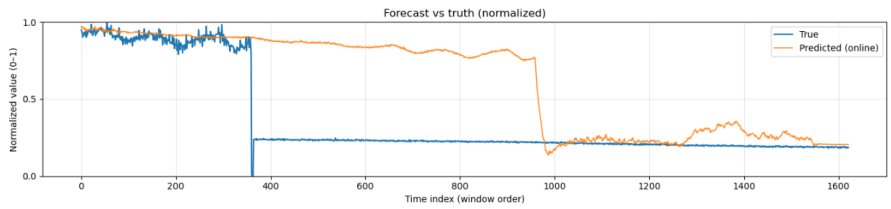

Figure 19: MLP online

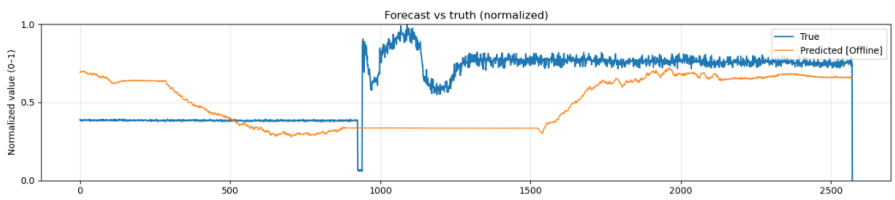

Figure 20: MLP offline

Online learning with a replay buffer achieves better forecasting performance than online learning without a replay buffer. Across the evaluated rolling blocks, replay-based updates provide more consis-

tent error reductions. Overall, the replay buffer improves online adaptation in this setting.

[1] Qinglin Cai, Leiying Chen, Jian Shao, and Ling Chen. Data-adaptive dynamic time warping-based multivariate time series fuzzy clustering. *IEEE Access*, 13:75525–75534, 2025. doi: 10.1109/ACCESS.2025.3564539.

[2] Ching Chang, Chiao-Tung Chan, Wei-Yao Wang, Wen-Chih Peng, and Tien-Fu Chen. Timedrl: Disentangled representation learning for multivariate time-series. In *Proceedings of the 40th IEEE International Conference on Data Engineering (ICDE)*. IEEE, 2024. Published in IEEE ICDE 2024.

[3] Lin Fang, Hongjie Li, Yingchao Zheng, and Xinggang Luo. Driving behavior recognition and fuel economy evaluation for heavy-duty vehicles. *Research in Transportation Business & Management*, 60:101371, 2025. doi: 10.1016/j.rtbm.2025.101371.

[4] Bulat Khaertdinov, Esam Ghaleb, and Stylianos Asteriadis. Contrastive self-supervised learning for sensor-based human activity recognition. In *2021 IEEE International Joint Conference on Biometrics (IJCB)*, pages 1–8, 2021. doi: 10.1109/IJCB52358.2021.9484410.

[5] Jing Liu, Yang Liu, Wei Zhu, Xiaoguang Zhu, and Liang Song. Distributional and spatial-temporal robust representation learning for transportation activity recognition. *Pattern Recognition*, 140:109568, 2023. doi: 10.1016/j.patcog.2023.109568.

[6] Emilia M. Szumska, Łukasz Pawlik, Damian Frej, and Jacek Łukasz Wilk-Jakubowski. Machine learning applications in energy consumption forecasting and management for electric vehicles: A systematic review. *Energies*, 18(20):5420, 2025. doi: 10.3390/en18205420.

[7] Obed Tshimanga Tshipata, Vincenzo Paciello, Dave Tshimbalanga Kazumba, Pierre Sedi Nzakuna, and Angelo Kuti Lusala. Multi-horizon short-term electrical load forecasting: A comparative analysis of statistical models and deep neural networks. In *2024 IEEE International Symposium on Measurements & Networking (M&N)*, 2024. doi: 10.1109/MN60932.2024.10615368.

[8] Zhenkan Wang, Yuantao Fan, Henrik Ydreskog, and Slawomir Nowaczyk. Investigation on machine learning models for forecasting auxiliary energy consumption of hd-bevs. In *Proceedings of the 38th International Electric Vehicle Symposium & Exhibition (EVS38)*, Göteborg, Sweden, 2025.

[9] Daoping Wu, Suhas Gundimeda, Shaoshuai Mou, and Christopher J. Quinn. Unsupervised change point detection in multivariate time series. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2024. PMLR, Volume 238.

[10] Yang Xing, Chen Lv, Dongpu Cao, and Chao Lu. Energy oriented driving behavior analysis and personalized prediction of vehicle states with joint time series modeling. *Applied Energy*, 261:114471, 2020. doi: 10.1016/j.apenergy.2019.114471.

[11] Xiaofan Yu, Yunhui Guo, Sicun Gao, and Tajana Rosing. Scale: Online self-supervised lifelong learning without prior knowledge. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 2484–2495, 2023. doi: 10.1109/CVPRW56620.2023.00937.

.

HALMSTAD UNIVERSITY

PO Box 823, SE-301 18 Halmstad
Phone: +35 46 16 71 00
E-mail: registrator@hh.se
www.hh.se